BroStatus v1.0.0 by DiamondSash

The status display for Claude Code: A diamond shows you with one color whether Claude is working, waiting for you or done. Next to it you see context, prompt cache, tools used and files changed, plus answer buttons above the input field.

As of 4 Nov 2026 · Version 1.0.0 · tested with Claude Code 2.1.310 · Windows | Mac | Linux · no servers of its own

Where it runs: only in Claude Code – in the terminal (`claude`) on Windows, Mac and Linux, in the Claude desktop app under "Code" and in editors like VS Code with the Claude Code extension. **Tested** (my own tests): Windows 10 with the Claude desktop app in real use; I have not yet looked at the edition without the automatic features in the real interface; it shares its code with the full edition, which I tested on 30 Oct 2026 in the terminal (PowerShell) and the desktop app (diamond, panel), VS Code on Windows (29 Oct 2026: panel, status, currency) and the installer on a second Windows PC. Mac: tried by one person in the terminal, the Mac desktop app is not tested yet. Linux: the installer is included, not yet tested in real use, feedback is welcome. It does not run in the regular Claude chat or in other AI tools (for example Cursor, Codex, Gemini).

5 colors	3 ways	0 tokens
of the diamond show you at a glance whether Claude is working, waiting for you or done	to reach you: window title, terminal notification, Telegram to your phone	is what the display itself costs (a token is the unit in which Claude counts and bills text)

Not included in this edition: keep-warm, Auto-Compact, Feierabend and model switch. These features send requests through your account on their own. Whether that fits the terms of Anthropic's plans has been asked of Anthropic and is not answered yet; they will come only once that is clarified.

What you get out of it

What BroStatus takes off your hands.

When you step away

The diamond turns gray as soon as Claude is waiting for you. On longer runs your computer or your phone lets you know.

The diamond · Signals

Cache and costs

You see the cache countdown. If the cache is cold, you see it and know why the next message costs more. Send a message in time yourself.

Cache row

See the context

The traffic light warns you in good time before the context is full. Then you summarize the conversation yourself with `/compact`.

Context traffic light

Answering questions

One button per item, type the answer, send everything together. Claude gets each answer with the matching question.

Answer buttons

What Claude is doing

BroStatus shows live which tools are running and which files are changed, new or only read.

Active and files

Example: cache

You are working on something, get distracted and forget the chat. The cache, the temporary store for the part of the conversation Claude has already read, expires after a pause. BroStatus shows the countdown. If the cache is cold, you see it and know why the next message costs more: Claude has to read everything again. Your conversation is not lost. If you want to avoid the extra cost, send a message in time yourself.

Example: compact

You code in a flow and do not see the context. The fuller it gets, the more accuracy and recall degrade (often called "context rot"). The context bar shows how full it is, and the traffic light warns you. Then you summarize the history yourself with `/compact`. What is in your files stays; what was only in the chat is afterwards only in the shorter summary. In my experience, the better the rules in your project, the less is lost.

Example: sounds

You are doing something else while Claude works. When Claude asks for an approval or shows a choice, BroStatus plays a warning sound in the desktop app, and you can come back and respond. When Claude is done after a longer run, a quiet sound plays. You set both under "More" or switch them off. In the terminal the terminal bell sounds instead.

Local and free: The display runs without servers of its own, sends nothing to the author and uses no tokens. Only this leaves your computer: Telegram (only if you set it up yourself) and, on Windows, the Python download through winget (the installer accepts the terms of winget and Python for you). The `state/` folder holds local status files with file paths and the first characters of commands. BroStatus itself sends no requests through your account.

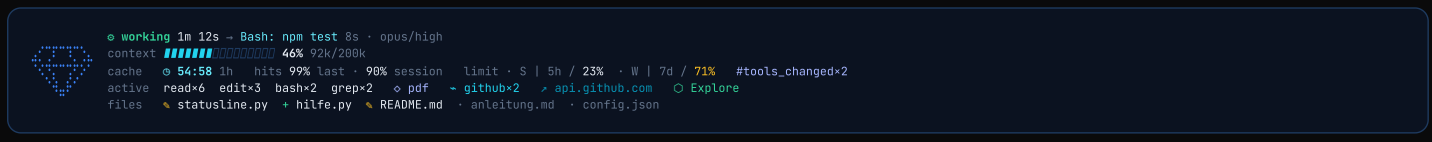
Contents

All status line images come straight from BroStatus itself.

01 At a glance
02 The diamond
03 Status row
04 Context traffic light
05 Cache row
06 Active and files
07 Answer buttons
08 The /bro panel
09 Done and waiting signals
10 Installation
11 Check that everything works
12 Settings
13 Updates and fixing problems

At a glance

This is what BroStatus looks like at the bottom of Claude Code while Claude is working.



Real status line output with sample data: Claude has been working for 1 minute 12 seconds and is running tests right now.

DIAMOND	The logo is on the left. Its color shows in one second whether you are needed. More
STATUS	What Claude is doing right now, for how long, which tool is running, plus the model and thinking level. More
CONTEXT	How full the conversation's memory is, with a traffic light and a tip on when to tidy up. More
CACHE	How long the prompt cache stays warm, how well it hits and how much of your limits is used. More
ACTIVE	Which tools, skills, MCP servers, web addresses and subagents were used in this session. More
FILES	Which files Claude has changed, created or only read. More

The diamond

Always exactly one color. If several apply, the most important wins.



Red · critical

API error, context at 85 % or more, or the cache expires within the next 2 minutes (1-h cache) or 30 seconds (5-min cache).



Gray · your turn

Claude is waiting for your approval or answer. Light silver, so it stands out.



Orange · act soon

Context at 50 % or more, or the cache expires soon (1-h cache: under 10 minutes, 5-min cache: under 1 minute).



Blue · working

Claude is in the middle of an answer. You do not have to do anything.



Green · done

The answer is here, everything is fine. Also green when idle and "ready".

Priority: red › gray › orange › blue › green

Example: Claude is working, but the context is at 58 %. Then the diamond is orange instead of blue, because "act soon" matters more than "working". You change the context limits with `ctx_stages` in the settings.

Status row

The first row tells you what Claude is doing right now.

READY

Nothing has happened yet in this session. On the right are the model and thinking level, here Opus with "high".

```
○ ready · opus/high
```

WORKING

Claude has been working for 1 minute 12 seconds. After the arrow comes the running tool with its run time. If several run at once, "+2 parallel" follows.

```
⚙ working 1m 12s → Bash: npm test 8s · opus/high
```

THINKING

Claude is working but is not using a tool right now.

```
⚙ working 5s → thinking… · opus/high
```

WAITING

Claude needs you, usually for an approval. Behind it you see what Claude is waiting for. The diamond turns gray.

```
‖ waiting for you – Bash: git push · opus/high
```

DONE

The answer is here: finished 12 seconds ago, after 3 minutes 20 seconds of work.

```
✓ done 12s ago · ran 3m 20s · opus/high
```

IN THE BACKGROUND

Claude has answered, but a command or helper agent is still running, two here. Instead of “done” it shows “working in the background” with the first task, and the diamond stays blue. In the panel there is one line per task below it. Only when everything has finished does “done” appear, with the sound.

```
◔ working in the background · 2 → npm run build +1 · 40s ago · opus/high
```

CANCELED

The answer was canceled by an error in the interface. Usually it helps to send the message again.

```
× canceled (API error) 3s ago · opus/high
```

Context traffic light

How full the conversation’s memory is, with a tip on what to do now.

UNDER 50 %

Cyan: all relaxed. After the percentage you see the tokens used and the tokens possible.

```
context ▮▮▮▮▯▯▯▯▯▯▯▯▯▯▯▯ 30% 60k/200k
```

FROM 50 %

Yellow: **finish the task**. Complete the current task before you start something new.

```
context ▮▮▮▮▮▮▮▮▯▯▯▯▯▯▯▯ 58% 116k/200k ▸ finish the task
```

FROM 70 %

Orange: **/compact or new chat**. Now is a good moment to tidy up.

```
context ▮▮▮▮▮▮▮▮▮▮▮▮▯▯▯▯ 75% 150k/200k ▸ /compact or new chat
```

FROM 85 %

Red: **Compact soon!** Claude Code will soon compact the conversation on its own.

```
context ▮▮▮▮▮▮▮▮▮▮▮▮▮▮▯▯ 88% 176k/200k ▸ Auto-Compact soon!
```

GOOD MOMENT

If the cache is already cold and Claude is not busy, a `/compact` costs hardly more than the rebuild that is due anyway. Then the green hint appears.

```
context ▮▮▮▮▮▮▮▮▮▮▮▮▯▯▯▯ 75% 150k/200k ▸ /compact or new chat (cache cold → good moment)
```

Cache row

The prompt cache saves money and quota for as long as it is warm.

Your benefit: You know how long you can take a break without your next message becoming expensive.

Claude Code caches the earlier history of a conversation at Anthropic for a while. As long as this cache is warm, the history is only read when you send your next message, and that costs about a tenth (less on some models). Once it has expired, everything has to be written again.

WARM

The countdown shows how long the cache will last, next to it the duration (`1h` or `5m`).

```
cache   ◔ 54:58 1h   hits 99% last · 90% session   limit · S | 5h / 23%  · W | 7d / 71%   #tools_changed×2
```

EXPIRES SOON

Yellow as soon as less than 10 minutes are left on the 1-h cache. With the 5-min cache there is no yellow here, only in the panel.

```
cache  ◷ 8:05 1h · expires soon   hits 99% last · 90% session   limit · S | 5h / 23%  · W | 7d / 71%   #t…
```

EXPIRES ANY MOMENT

Red in the cache row: in the last 2 minutes, with the 5-min cache in the last minute (the diamond and the panel turn red only in the last 30 seconds with the 5-min cache). If you want to carry on, send your message now.

```
cache  ◷ 1:34 1h · expires soon   hits 99% last · 90% session   limit · S | 5h / 23%  · W | 7d / 71%   #t…
```

COLD

Expired. “new ~92k” means: your next message writes about 92,000 tokens into the cache again.

```
cache  ◷ cold 1h · new ~92k   hits 99% last · 90% session   limit · S | 5h / 23%  · W | 7d / 71%   #tools…
```

5-MIN CACHE

Depending on your account and settings, the cache may last only 5 minutes (often with an API key or extra credit).

```
cache  ◷ 3:31 5m   hits 99% last · 90% session   limit · S | 5h / 23%  · W | 7d / 71%   #tools_changed×2
```

Hits

- **last:** share of the last request that came from the cache
- **session:** the same for the whole session
- High is good. 99 % means almost everything was read cheaply

Limit

- Use of your plan: **5-hour window / week**
- White up to 64 %, yellow from 65 %, red from 85 %

#Reasons

- Why the cache last had to be rewritten
- `#tools_changed×2` : The tool list changed twice

Active and files

What was used and touched in this session.

```
active  read×6  edit×3  bash×2  grep×2  ◇ pdf  ⌁ github×2  ↗ api.github.com  ⬡ Explore
```

Symbols in the active row

- **no symbol:** tools like read, edit, bash, with a count (`read×6`)
- ◇ skills that Claude has loaded
- ⌁ MCP servers, meaning connected services
- ↗ web addresses and web search
- ⬡ subagents that helped out
- At most 5 entries per kind, the rest as `+3`

Symbols for files

- \+ written (new or overwritten)
- ✎ changed
- · only read
- Changed files come first, newest first

```
files  ✎ statusline.py  + hilfe.py  ✎ README.md  · anleitung.md  · config.json
```

Compact view (default): one row, whatever does not fit appears as `+2` at the end.

```
├─ docs/
│  └─ · anleitung.md
├─ src/
│  ├─ neu/
│  │  └─ + hilfe.py
│  ├─ · config.json
│  └─ ✎ statusline.py
└─ ✎ README.md
```

With `"files": "tree"` a file tree with up to 8 rows appears below.

Answer buttons

Mod “bro-status”: answer open items without typing everything out.

If Claude’s answer ends with a numbered list or bullet list that wants something from you, a band with one button per item appears above the input field. You answer the items one by one, and “Send” sends everything together as your message.

```
◆ Answers  1/2 answered  [ Send (1) ]  [ Hide ]
[ 4 images are still missing ]   ✓ already in the folder
[ You still need to do this ]
On 2 is done                                  accept
```

Recreated from the mod’s code. In Claude Code the band looks slightly different depending on your terminal.

How it works

1. Click the button for the item
2. Type the answer into the “Re 1” field, `Enter`. It jumps to the next open item
3. “Send”: all answers go out together
4. “Hide” hides the band until the next answer

What arrives

Each answer becomes one line with the number and the item in parentheses, so Claude knows exactly what you are referring to:

```
Re 1 (4 images are still missing):
already in the folder
Re 2 (You still need to do this): is
done
```

Good to know

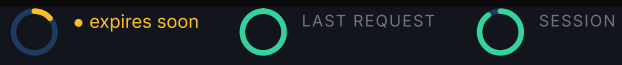
- The last list with at least 2 items counts, with at most 9 buttons
- Only for questions: a “?” is in an item, in the line directly before or in the paragraph directly after (for bullet lists with “-” only the paragraph after counts). A heading right before it such as “What I need from you:” or “Open questions:” counts too. Words like “Let me know” alone trigger no band, nor do plain instructions and lists
- An open question without a list (“How should I do this?”, “Left or right?”) gets no band; answer it directly in the input field.
- A yes/no question at the end (for example “Does the plan work for you?”) gets two buttons, **YES** and **NO**. One click sends “Yes” or “No” right away, even with a list before it
- If Claude names ways as separate lines (“Way 1 (recommended): …”, “Way 2: …”) and asks openly (“How should I do this?”), there is one button per way, **WAY 1**, **WAY 2**; the recommendation is marked and a click sends “Way 1” right away
- Under a report without a question there are the buttons **NEXT TASK**, **SAVE** and **WHAT IS OPEN?** after a finished task, and only **WHAT IS OPEN?** for a progress note; short answers get nothing. Detection works on words like “checked”, “green” or “next up”. They send “Show me the tasks that can start now.”, “Save now: check, commit, pull request.” and “What is still open for this task?”. They are meant for projects with a task list and Git
- Long items are shortened at a word end, the full question appears in the answer field
- Lists inside code blocks are skipped
- Without a mouse: `Ctrl+X`, then `Tab` to the buttons and `Enter`. `Esc` goes back to the input field
- The buttons deliberately have no keyboard shortcuts: nothing gets triggered by accident while you type
- Every new message clears the band

The /bro panel

You open the panel with `/bro`. In the desktop app it opens by itself with your first message.

Your benefit: Even in the desktop app, where there is no status line, you see everything at a glance.





Recreated from the mod’s code, the way the desktop app shows it. At the top is the status with the diamond, below it the cache. In the terminal, the remaining time and hit rates appear as text instead of circles. The Sounds row: name and state on the left, the switch on the right. The fine settings are under “More”.

STATUS AT THE TOP	The same information and colors as the status line: diamond, what Claude is doing, context traffic light, limits and cost, tools and files used. Meant for the desktop app; in the terminal it is a summary of the status line.
WAITING FOR YOU	If Claude Code asks you for an approval or Claude shows you a choice to click, “waiting for you” appears in red right away with what is to be approved, and in the desktop app the alert sound plays. In auto mode the auto checker looks first: while it checks, “Auto checks: <tool>” is shown and it stays quiet. “waiting for you” only comes when you are really asked. BroStatus detects the approval through the hooks in `settings.json` ; if they are missing, “waiting for you” appears after 30 seconds without a start. The desktop app does not report the approval back: the display stays until the tool is finished. After 30 seconds it therefore adds “approval or running for …” with a counting-up time and, shortly before and after the cache expires, a cache warning.
CACHE ESTIMATED	If the status line is running, the cache countdown comes exactly from its data. In the desktop app without a status line, BroStatus estimates 1 hour from the last request and adds “estimated”. The hit rates then come from Claude’s answers.

BUTTON / FIELD	EFFECT
◆▾ / ◆▸ (left, before the circles)	Hides or shows the switches and More so the panel takes up less space. Status and circles stay. Is remembered.
More ▸ / More ▾	Opens up, in sections: the three circles and the limits, Sounds, Language, Cost and tip
Language (under More)	Deutsch, English, Español, Italiano or Français. Applies to the panel, answer band, messages and the status line in the terminal; your choice stays saved.
Currency (under More)	Dollar, euro, pound or franc. The API value in the “cost” row is converted with a built-in rate (ECB reference rate, the date is shown under the selection). You enter your own rate in `config.json` (`currency_rate` : how many units of the currency one dollar is). It stays an estimate (≈), not an invoice amount. The rates are in `kurse.json` and are updated with new versions; BroStatus does not fetch them from the internet.
Close	`/bro` once more or `Esc` , in the desktop app the cross on the panel. Everything you switched on keeps running
· S | 5h / … · W | 7d / … (row “limit”)	S is the session limit (the current 5 hours), W the weekly limit (7 days), each showing how much of it is used. Yellow from 65 %, red from 85 %. The same in the status line.
· API | $ (row “cost”)	API is the estimated API value of the session (with a plan you do not pay it). The sign ≈ means: estimate, not an exact value. For cache reads BroStatus uses the factor of the model you are using (according to Anthropic’s price list, as of 29 Oct 2026: 0.05 for Opus 5.5 and Sonnet 5.5, 0.025 for Fable 5.1 and Mythos 5.1, otherwise 0.1). If BroStatus does not know the model it uses 0.1; for new models the factor may differ.
Sounds (desktop app only)	Its own row with a light (**green** on, **red** off) and On/Off, on by default: a noticeable sound for “waiting for you”, a quiet sound for “done” after longer work. Under “More” you choose one of three sounds per occasion, “Done sound from duration” and “Repeat sound” (Off, every 2 or every 5 minutes): a missed sound comes back at most 3 times, until you write, approve or choose. With sample and test buttons. “Sound volume” has the steps 25, 50, 75 and 100 % (default 50 %) and applies to all sounds, including your own; it works on Windows, while on Mac and Linux the app plays the sound at its own volume. In the terminal, the terminal bell stays.
· next message ≈ … tokens (under “context”, panel only)	What the next message costs according to the cache state: *warm* about a tenth (less on some models) of the context, *cold* the rebuild. An estimate; without cache data, no number.

Colors in the panel

- ● **warm**: all good, the circle shows the remaining time.
- ● **expires soon**: under 10 min (1-h cache) or 1 min
- ● **expires any moment**: under 2 min or 30 s, ● **cold**: expired
- Rings: green from 80 %, yellow from 50 %, otherwise red

Claude Code compacts by itself after about 1 hour of idle time; switch that off with `"idleCompaction": false` in `settings.json`. Set the cache duration yourself: `"promptCacheTtl": "1h"`.

Done and waiting signals

So you do not have to keep checking.

Window title

The title of the terminal window shows the status, even when it is in the background:

```
◆ my-project · ⚙ Bash npm test
◆ my-project · ⏸ waiting for you
◆ my-project · ◔ working in the background (2)
◆ my-project · ✓ done (3m 20s)
```

Sound and terminal notification

When Claude is waiting for you, or when an answer took at least 15 seconds:

```
💎 ✓ Claude is done – my-project (3m 20s)
Claude is waiting for you (my-project)
```

You set the threshold with `notify_min_secs`.

Telegram via your own bot

Optional. Handy if you control Claude from your phone. The message contains the project name and the notice (done with duration, or waiting for you). BroStatus only adds the start of Claude's answer (400 characters) and the computer name if you switch them on in `config.json` (`telegram_include_answer`, `telegram_include_host`, off by default).

Setup: see Settings.

Want to use your own sounds?

Works on Windows in the desktop app. Your sounds stay with you, an update does not delete them.

1. In the BroStatus folder `C:\Users\<Name>\.claude\brostatus` (next to `state`) create a folder `toene`.
2. Put your sounds in it as WAV files: `warten–<Name>.wav` for the warning sound, `fertig–<Name>.wav` for the done sound, for example `fertig-Gong.wav`.
3. Start a new session. Under `/bro` → "More" → Sounds, "Gong" appears after the built-in sounds. The first time, your sound becomes the default right away.

Only use sounds you are allowed to use. On Mac and Linux, BroStatus only plays the built-in sounds.

Installation

Windows | Mac | Linux. One installer per system. It sets up the status line, signals and mod and makes a backup first.



You recognize the installers by this icon. On Windows the `.exe` has this icon, on the Mac the app. After installation, the `brostatus` folder has this icon too.

Windows



BroStatus-Install.exe
Double-click is enough

1. Download `BroStatus-Install.exe`
2. Double-click it
3. At "Windows protected your PC": **More info**, then **Run anyway**
4. If Python is missing, the window installs it by itself (1–2 minutes)
5. Choose a folder or press `Enter`
6. At "✓ Fertig!" (German for "Done") press any key
7. Restart Claude Code, preferably in **Windows Terminal**

As a fallback there is `BroStatus-Install.cmd`, which does the same. It shows the normal CMD symbol.

Mac



BroStatus-Install.app
in `BroStatus-Mac.zip`

1. Download `BroStatus-Mac.zip` and unpack it with a double-click
2. The first time, **right-click** `BroStatus-Install.app` → **Open** → **Open**
3. No Open button? **System Settings** → **Privacy & Security** → **Open Anyway**
4. If Python is missing: in the terminal `xcode-select --install`, then try again
5. At "✓ Fertig!" (German for "Done") press `Enter` and restart Claude Code

Linux

brostatus-install.sh
Script for the terminal, without its own icon

1. Copy `brostatus-install.sh` to the computer, e.g. from your PC (NAME = your user, SERVER = the server's name or IP address):

```
scp brostatus-install.sh NAME@SERVER:~/
```

2. Start it:

```
bash brostatus-install.sh
```

3. If Python is missing: `sudo apt install -y python3`, then try again
4. Restart Claude Code

If you use Claude Code on a server over SSH, you only see the status line in the terminal. If you control it from your phone over Remote

Control, you get the Telegram message instead.

FOLDER — The default is `~/.claude/brostatus`, on Windows `C:\Users\<Name>\.claude\brostatus`. If you give a different folder, everything goes into the subfolder `brostatus` inside it, and updates stay there. On Mac and Linux: `--ziel <ordner>`.

SETTINGS — Claude Code's `settings.json` normally stays in `~/.claude` (or in the folder given by `CLAUDE_CONFIG_DIR`). The installer adds the status line, hooks and mod there and makes a backup first.

Check that everything works

In three steps after the restart.

Start Claude Code first, then /bro: `/bro` and `/plugin` are commands *inside* Claude Code, not in PowerShell or the terminal. Open the terminal normally (not as administrator), change to the project folder (`cd D:\mein-projekt`), type `claude`. On the first start, Claude Code signs you in once in the browser. In the desktop app, just start a new session.

1 · STATUS LINE — The diamond, "○ bereit" and the info rows appear at the bottom, in German at first. Under `/bro` → More → Language you switch to English, then it looks like this:



```
○ ready · opus/high
context ▮▮▮▮▮▮▮▯▯▯▯▯▯▯▯▯▯ 46% 92k/200k
cache   ◷ 54:58 1h   hits 99% last · 90% session   limit · S | 5h / 23%  · W | 7d / 71%   #tools_changed×2
active  –
files   –
```

2 · MOD LOADED — Type `/plugin`. Under the tabs it says "1 mod active · bro-status" (a higher number if you have other mods of your own).

```
> /plugin
1 mod active · bro-status
```

Excerpt, recreated

3 · BUTTONS — Ask Claude to put two questions to you as a numbered list. "◆ Answers" appears above the input field.

```
◆ Answers  press a button  Send  Hide
Pick a color
Logo left or right
```

Excerpt, recreated

Settings

In `~/.claude/brostatus/config.json`. Changes take effect immediately, without a restart (only your own rate is read when the mod starts).

KEY	DEFAULT	MEANING
`notify_min_secs`	15	After this many seconds of work, a done message is sent
`terminal_notify`	true	Sound and terminal notification
`window_title`	true	Status in the window title
`telegram_bot_token`	empty	Your bot's token for Telegram messages
`telegram_chat_id`	empty	Your chat ID for Telegram
`logo`	true	Show the diamond on the left
`colors`	"truecolor"	Set to "256" if colors look wrong
`ctx_stages`	[50, 70, 85]	Limits of the context traffic light in % (yellow, orange, red)
`files`	"compact"	"compact" = one row, "tree" = file tree, "off" = off
`tree_max_lines`	8	At most this many rows for the file tree
`max_items`	5	Entries per kind in the active row

Set up Telegram with your own bot

1. In Telegram, create your own bot with `@BotFather` and note the token
2. Send your bot any message in Telegram
3. Open this in the browser, replacing `<TOKEN>` with the bot token: `https://api.telegram.org/bot<TOKEN>/getUpdates`
4. Your chat ID is under `"chat":{"id": …}`
5. Enter the token and chat ID in the `config.json` :

```
"telegram_bot_token": "123456:ABC...",
"telegram_chat_id": "your-chat-id"
```

Updates and fixing problems

For an update, just start the new installer file. Your `config.json` is kept.

PROBLEM	SOLUTION
`claude` is not recognized	Claude Code is installed, but not in the PATH. Run this once in PowerShell, then reopen PowerShell: `[Environment]::SetEnvironmentVariable('Path', [Environment]::GetEnvironmentVariable('Path','User') + ";$env:USERPROFILE\.local\bin", 'User')`
`/bro` is not recognized (red message in PowerShell)	You are still in PowerShell. Start `claude` first, then type `/bro` in Claude Code
In the desktop app, `Enter` after `/bro` picks a different suggestion (e.g. a skill)	After `/bro` , the suggestion menu only shows skills, not the mod command. Press `Esc` first, then `Enter` . In a fresh session the mod only starts with your first message
"Browser didn't open? … Paste code here"	One-time sign-in to Claude Code: open the link, sign in with your Claude account, paste the code shown into the terminal (right-click), `Enter` . Do not share the code anywhere else
Claude Code starts in `C:\WINDOWS\system32`	Do not open PowerShell as administrator, and change to your project folder with `cd` first
Boxes instead of the diamond	Use Windows Terminal instead of the old cmd window, or set `"logo": false`
Colors look odd	Set `"colors": "256"` in the `config.json`
No status line	Restart Claude Code and trust the folder (confirm the trust prompt)
`/plugin` does not show bro-status	Check the version with `claude --version` , 2.1.310 is tested (older versions not checked). Otherwise `claude update`
Reaching the buttons without a mouse	`Ctrl+X` , then `Tab` and `Enter`
Python is missing on Windows	Install it from python.org, tick "Add python.exe to PATH", start the installer again
Python is missing on Linux	`sudo apt install -y python3`
"Da ist etwas schiefgelaufen" ("Something went wrong")	Paste a screenshot into your Claude Code session and ask for help, or report it as an issue

Test the status line on its own

If the diamond appears with "○ ready" (or "○ bereit"), BroStatus is fine and the problem lies with Claude Code.

```
echo '{"session_id":"test","model":{"display_name":"Opus"}}' |
python3 ~/.claude/brostatus/statusline.py
```

Uninstall

Mac and Linux:

```
python3 ~/.claude/brostatus/install.py --uninstall
```

Windows (PowerShell):

```
python $HOME\.claude\brostatus\install.py --uninstall
```

Removes BroStatus's entries from `settings.json` (a backup is made first). The `brostatus` folder stays, with your `config.json` (which holds your Telegram token, if you set it up). Delete it by hand when you no longer need it.

BroStatus is a project by DiamondSash (github.com/diamondsash/bro-status). Made for Claude Code, no servers of its own and uses no tokens for the display. License: MIT with Commons Clause (free to use, not for sale). Not affiliated with Anthropic. Claude and Claude Code are trademarks of Anthropic.